

Pay-Together — 100 FYP Viva Voce Questions & Answers

Project Title: Pay-Together: Collaborative Expense Management, Spending Limit Alerting, and Debt Settlement System for Group Travel

Technology Stack: Python, Django 6.0, Django REST Framework (DRF), JavaScript (ES6+), SQLite / PostgreSQL, HTML5/CSS3 (Tailwind CSS), Chart.js

Table of Contents

1. [Category 1: Project Overview, Scope & Problem Formulation (Q1 – Q10)]
(#category-1-project-overview-scope--problem-formulation)
2. [Category 2: Requirements Engineering & SDLC Methodology (Q11 – Q20)]
(#category-2-requirements-engineering--sdhc-methodology)
3. [Category 3: System Architecture & Design Patterns (Q21 – Q30)](#category-3-system-architecture--design-patterns)
4. [Category 4: Database Design, ORM & Data Integrity (Q31 – Q40)](#category-4-database-design-orm--data-integrity)
5. [Category 5: Core Algorithms & Mathematical Logic (Q41 – Q50)](#category-5-core-algorithms--mathematical-logic)
6. [Category 6: Frontend Engineering, State & Offline Sync (Q51 – Q60)](#category-6-frontend-engineering-state--offline-sync)
7. [Category 7: Security, Authentication & Access Control (Q61 – Q70)](#category-7-security-authentication--access-control)
8. [Category 8: Software Testing, Quality Assurance & Metrics (Q71 – Q80)]
(#category-8-software-testing-quality-assurance--metrics)
9. [Category 9: Performance Optimization & Deployment (Q81 – Q90)](#category-9-performance-optimization--deployment)

10. [Category 10: Critical Defense, Justification & Future Scope (Q91 – Q100)]
(#category-10-critical-defense-justification--future-scope)

Category 1: Project Overview, Scope & Problem Formulation

Q1: What is the primary purpose and motivation behind "Pay-Together"?

Answer: The purpose of Pay-Together is to automate shared financial tracking, spending limit surveillance, and debt settlement during group excursions. In conventional trips, individuals pay for food, transport, and lodging haphazardly using cash, cards, and digital wallets. Reconstructing these expenses after the tour results in arithmetic errors, lost paper receipts, circular debt confusion, and interpersonal friction. Pay-Together centralizes these interactions into collaborative tour workspaces with real-time balance calculations and simplified settlements.

Q2: What are the core modules of the Pay-Together platform?

Answer: The system is composed of nine interconnected modules:

1. User Identity & Authentication (Custom User model, RBAC)
2. Tour Workspace & 6-Character Join Token Management
3. Collaborative Expense Ledger (Multi-payer, flexible split options)
4. Custom Share Splitter (Equal, exact amounts, percentages)
5. Digital Receipt Vault (Image storage, lightbox preview, verification audits)
6. Personal Spending Limits & Edge-Triggered Alerts
7. Client-Side Offline Queue & Idempotent Sync
8. Greedy Debt Settlement Engine (Debt minimization)
9. Analytical Reports & Administrative Surveillance Gateway

Q3: What fundamental problem does Pay-Together solve that generic apps like WhatsApp or Excel cannot?

Answer: Generic spreadsheets and messaging apps lack atomic financial consistency, automated split logic, and debt-graph minimization. Spreadsheets require manual formula management which fails on mobile devices in transit, while chat apps bury transactions under messages with no audit trail. Pay-Together guarantees ACID transaction integrity, automated greedy debt simplification (reducing $\$O(N^2)$ bilateral debts to at most $\$N-1$ transactions), receipt verification, and offline synchronization.

Q4: Why is group expense tracking during travel particularly vulnerable to disputes?

Answer: Group travel expenses are asynchronous, multi-currency, and varied (some meals are shared by everyone, while some cab rides include only three members). Without an immediate ledger with receipt attachments, travelers rely on memory days later, leading to recall bias, missing cash payments, and disputes over who opted into which activity.

Q5: What is the functional scope and boundaries of the system?

Answer: The scope encompasses user account management, tour lifecycle management (Planning, Active, Completed, Archived), flexible expense splitting, receipt storage, spending limit alerts, analytical dashboards (spending by category and member), and debt calculation. The current scope deliberately excludes direct banking/fiat payment gateway integration (e.g., executing actual bank transfers), relying instead on recorded settlement proofs and external bank verification.

Q6: How does the Tour Join Token system work?

Answer: Instead of tedious email invitations, tour creators generate a secure, human-memorable 6-character alphanumeric token (e.g., **TRV902**). Travelers simply enter this token or scan a QR code URL to instantly join the tour workspace as an active participant.

Q7: What are the primary user roles in Pay-Together?

Answer: The platform implements three distinct roles:

1. **System Administrator:** Oversees system health, user status, security logs, and platform analytics via a secured administrative portal.
2. **Tour Organizer / Creator:** Manages tour settings, invites/removes members, sets overall budgets, and can finalize settlements.
3. **Tour Member:** Logs personal or shared expenses, views workspace ledgers, uploads receipts, sets personal spending ceilings, and resolves balances.

Q8: What happens when a tour status changes from "Active" to "Completed"?

Answer: In the "Active" state, members can freely add, edit, and delete expenses. When marked "Completed", the ledger becomes read-only to freeze financial records, allowing the settlement engine to compute final net balances and generate official closure reports.

Q9: What are the target non-functional requirements (NFRs) of Pay-Together?

Answer:

- Sub-second server response time (< 500 ms for core APIs)
- 100% mathematical accuracy and zero balance discrepancy ($\sum \text{net balances} = 0$)
- Responsive UI compatible with desktop, tablet, and mobile browsers
- Resilient offline transaction queuing in remote travel zones
- Robust security against OWASP Top 10 vulnerabilities (CSRF, XSS, SQLi).

Q10: How does Pay-Together handle runaway spending in specific categories?

Answer: The platform features automated budget alerts. If an expense category (e.g., Alcohol, VIP Dining, or Luxury Transport) consumes more than 30% of the entire tour budget, the system triggers a warning badge on the dashboard to notify the organizer.

Category 2: Requirements Engineering & SDLC Methodology

Q11: Which Software Development Life Cycle (SDLC) model was adopted, and why?

Answer: The Agile Scrum methodology was employed. Group financial software requires continuous user feedback, incremental feature validation (such as split modes and offline sync), and rapid prototyping of complex mathematical logic (debt simplification). Sprints focused on deliverable vertical slices: Auth $\rightarrow$ Tours $\rightarrow$ Expenses $\rightarrow$ Settlement Engine $\rightarrow$ Analytics.

Q12: How were system requirements gathered and formulated?

Answer: Requirements were gathered through:

- Domain analysis of existing solutions (Splitwise, Tricount, Excel templates)
- Stakeholder interviews with frequent travelers, tour organizers, and university trip leads
- Use Case modeling, deriving 10 fully dressed use cases (UC-01 to UC-10)
- Functional Requirements cataloging (FR-01 to FR-15) and Non-Functional Requirements.

Q13: What is a Requirement Traceability Matrix (RTM), and why is it included in your documentation?

Answer: The RTM maps every business requirement to its corresponding software requirement, design element, code module, and test case (e.g., FR-08 Greedy Debt Settlement $\rightarrow$ `expenses.services.settlement` $\rightarrow$ Test Case `UT-06`). It ensures 100% test coverage and verifies that no requirement was omitted during development.

Q14: Mention three critical Functional Requirements (FRs) of Pay-Together.

Answer:

1. **FR-03 (Split Engine):** The system must compute splits across equal, exact, and percentage distributions with rounding compensation.
2. **FR-07 (Edge-Triggered Alerting):** The system must alert a user when an expense pushes their total share above their defined personal threshold.
3. **FR-08 (Debt Simplification):** The system must minimize total inter-member financial transfers to at most $N-1$ payments.

Q15: Mention three critical Non-Functional Requirements (NFRs) of Pay-Together.

Answer:

1. **Data Consistency:** Financial calculations must maintain 2 decimal places using arbitrary-precision arithmetic (`Decimal`), avoiding IEEE floating-point errors.
2. **Offline Durability:** Transactions recorded without internet access must be stored in browser storage (`localStorage` /IndexedDB) and synchronized upon reconnect.
3. **Security & Authorization:** Strict Object-Level Permission verification preventing users from accessing tours they are not enrolled in.

Q16: What is a "Fully Dressed Use Case"?

Answer: A comprehensive use case specification detailing: Use Case Name, Actor, Pre-conditions, Post-conditions, Trigger, Main Success Scenario (step-by-step), Alternative/Exception flows, and Frequency of Occurrence.

Q17: What is the difference between Verification and Validation in your project?

Answer:

- **Verification:** "Did we build the software right?" — Checked via automated unit tests, linter rules, code reviews, and schema migrations verifying adherence to architectural specs.
- **Validation:** "Did we build the right software?" — Checked by running end-to-end user travel workflows to ensure it genuinely solves the real-world group expense problem.

Q18: How was the Work Breakdown Structure (WBS) organized?

Answer: The WBS was structured hierarchically across 5 major phases:

1. Requirements & Feasibility Analysis
2. Architectural & Database Design
3. Implementation (Frontend, Backend, Calculation Services)
4. Verification & Testing (Unit, Integration, Security, Performance)
5. Deployment, System Conversion & Final Documentation.

Q19: What risks were identified during project planning, and how were they mitigated?

Answer:

- **Risk 1:** Floating point precision errors in division $\rightarrow$ *Mitigation:* Enforced Python's `decimal.Decimal` with round-half-up quantization.
- **Risk 2:** Duplicate submissions during offline sync reconnection $\rightarrow$ *Mitigation:* Implemented client-generated UUID idempotency keys.
- **Risk 3:** Unbalanced debt cycles $\rightarrow$ *Mitigation:* Enforced mathematical assertion that $\sum(\text{credits}) = \sum(\text{debits})$ before executing the greedy solver.

Q20: What is the purpose of the System Vision Statement?

Answer: The Vision Statement establishes the long-term target: To be the benchmark, zero-friction collaborative financial platform for group travel, eliminating mathematical ambiguity and financial disputes through transparent ledger accounting and automated debt resolution.

Category 3: System Architecture & Design Patterns

Q21: What architectural style does Pay-Together follow?

Answer: Pay-Together implements a modern Multi-Tier Client-Server Architecture utilizing Django's Model-View-Template (MVT) pattern augmented with RESTful API endpoints for asynchronous JavaScript interactions.

Q22: Explain the Django MVT (Model-View-Template) pattern used in this project.

Answer:

- **Model:** Represents the data structure and business constraints (e.g., `Tour`, `Expense`, `ExpenseSplit`), mapped to relational tables via Django ORM.
- **View:** Contains the business logic, request handling, permission enforcement, and orchestration of services (e.g., calling the settlement engine).
- **Template / Client View:** Renders the responsive HTML UI using Tailwind CSS, Jinja/Django template tags, and client-side JavaScript components.

Q23: Why is Django considered a "batteries-included" framework, and how did that benefit Pay-Together?

Answer: Django natively provides a secure ORM, cryptographic authentication, CSRF middleware, session management, automated schema migrations, and an administrative engine. This allowed us to focus on solving domain-specific problems (debt minimization, multi-payer splits, offline sync) instead of reinventing basic web infrastructure.

Q24: What design patterns are implemented in the Pay-Together codebase?

Answer:

1. **Service Layer Pattern:** Business logic (debt settlement, split calculations) is decoupled from views into standalone service modules (`services/settlement.py`).
2. **Repository/ORM Pattern:** Django ORM abstracts SQL queries behind expressive model managers.
3. **Strategy Pattern:** Split calculation employs strategies for Equal, Percentage, and Exact division.
4. **Observer / Signal Pattern:** Django signals trigger notification dispatch and edge-triggered budget threshold alerts upon expense creation.

Q25: Why did you separate business logic into a Service Layer rather than placing everything in Django Views or Models?

Answer: Fat views or fat models violate the Single Responsibility Principle (SRP) and make automated unit testing difficult. By placing algorithms (like the Greedy Settlement Engine) in dedicated service functions, they can be tested independently of HTTP request/response lifecycles and reused across web views, API endpoints, and CLI scripts.

Q26: What is the sequence of operations when a user logs an expense?

Answer:

1. User submits expense form (Payer, Amount, Category, Split Type, Receipt).
2. Django View validates CSRF token and verifies user's membership in the tour.
3. `atomic()` transaction begins.
4. `Expense` instance is saved.
5. `ExpenseSplit` records are generated based on chosen strategy.
6. Validation check: $\sum \text{splits} == \text{Total Amount}$.
7. Edge-triggered check: compares each member's cumulative spending against their personal ceiling; creates `Notification` if threshold is breached.
8. Database transaction commits, returning a JSON/redirect response.

Q27: How does Pay-Together handle asynchronous client-side requests?

Answer: Asynchronous interactions (such as receipt previews, dynamic split inputs, live budget charts, and offline sync) use JavaScript `fetch()` calls communicating with Django REST Framework (DRF) JSON API endpoints.

Q28: Describe the multi-tier deployment topology of the application.

Answer:

- **Tier 1 (Presentation):** Client web browser rendering HTML5/Tailwind CSS with Chart.js.
- **Tier 2 (Web/Application Server):** Nginx reverse proxy passing requests via WSGI/ASGI to Unicorn/Django application workers.
- **Tier 3 (Database & Storage):** Relational database (SQLite/PostgreSQL) and media storage for receipt attachments.

Q29: What is the role of the `core` or `cores` app in your Django project?

Answer: The `cores` app contains cross-cutting abstractions and utilities, such as `TimeStampedModel` (abstract base model with `created_at` and `updated_at`), shared template tags, global pagination helpers, and centralized error handlers.

Q30: What are Architectural Decision Records (ADRs), and which key decisions did you record?

Answer: ADRs document important architectural choices, their context, and consequences. Key ADRs in Pay-Together:

- **ADR-01:** Selection of Python/Django over Node.js/Express for robust transactional data modeling.
- **ADR-02:** Use of `DecimalField` instead of `FloatField` for all currency storage.
- **ADR-03:** Adoption of the Greedy Heuristic for debt settlement over NP-hard exact subset-sum solvers.

Category 4: Database Design, ORM & Data Integrity

Q31: Walk us through the primary database tables in Pay-Together.

Answer:

1. `User` : Custom user credentials and profile details.
2. `Tour` : Tour workspace, destination, dates, 6-character token, total budget, status.
3. `TourMember` : Junction table linking `User` and `Tour` with role (Organizer/Member) and personal spending limit.
4. `Expense` : The ledger item (Tour, Payer, Title, Amount, Category, Date, Receipt image).
5. `ExpenseSplit` : Breakdown detailing how much each member owes for a specific `Expense`.
6. `Settlement` : Recorded repayment transactions between two members.
7. `Notification` : Alerts for spending limits, invitations, and tour milestones.

Q32: Why did you use a Custom User model instead of Django's default `auth.User`?

Answer: Django official best practices strongly recommend subclassing `AbstractUser` at project inception. A custom user model allows adding custom fields (phone number, avatar, currency preference) and changing authentication to email-based login without needing disruptive database migrations later.

Q33: How is the relationship between Tours and Users structured?

Answer: It is a Many-to-Many relationship governed by an intermediate model (`TourMember`). Using `through='TourMember'` allows storing relationship-specific metadata: the member's role (Organizer vs Member), date joined, and custom personal spending threshold.

Q34: Why is `DecimalField` required for monetary values instead of `FloatField`?

Answer: Standard floating-point numbers (`float` / `double`) use binary base-2 representation (IEEE 754), which cannot accurately represent decimal fractions like 0.1 or 0.01, causing cumulative rounding errors (e.g., $\$0.1 + 0.2 = 0.30000000000000004\$$). `DecimalField` uses fixed-point decimal arithmetic, guaranteeing exact financial precision.

Q35: What database normalization level does the Pay-Together schema achieve?

Answer: The database achieves Third Normal Form (3NF):

- **1NF:** All fields contain atomic values, with primary keys on every table.
- **2NF:** All non-key attributes are fully functionally dependent on the entire primary key.
- **3NF:** No transitive dependencies exist; non-key attributes depend solely on candidate keys (e.g., member shares are factored into `ExpenseSplit` rather than serialized as comma-separated text in `Expense`).

Q36: How does Django ORM prevent SQL Injection attacks?

Answer: Django ORM automatically parameterizes all SQL queries. User input is never concatenated directly into raw SQL strings; instead, database drivers escape parameters automatically, neutralizing SQL injection vectors.

Q37: What is the "N+1 Query Problem", and how did you resolve it in Pay-Together?

Answer: The N+1 query problem occurs when fetching a list of parent records (e.g., 50 expenses) and then performing an additional database query for each record's related foreign key (e.g., `expense.payer` or `expense.tour`). This results in $1 + 50 = 51$ queries. In Pay-Together, we resolved this using `select_related('payer', 'tour')` for foreign keys (SQL `JOIN`) and `prefetch_related('splits')` for many-to-many relationships, reducing 50+ queries down to just 1 or 2 queries.

Q38: What are database transactions, and where are they strictly enforced in this system?

Answer: A database transaction guarantees ACID (Atomicity, Consistency, Isolation, Durability) properties. In Pay-Together, `transaction.atomic()` decorators are enforced during:

- Expense creation and split generation (if creating an `ExpenseSplit` fails, the `Expense` is rolled back).
- Batch offline sync synchronization.
- Settlement recording and balance recalibration.

Q39: What constraints exist on the `ExpenseSplit` table to maintain integrity?

Answer:

- `UniqueConstraint(fields=['expense', 'member'], name='unique_expense_member_split')` ensures no member is billed twice on the same expense.
- Non-negative value checks: `CheckConstraint(check=Q(amount__gte=0))` preventing negative debts.
- Application-level assertion verifying $\sum(\text{split amounts}) == \text{Expense.total_amount}$.

Q40: What happens in the database if a Tour or User is deleted?

Answer: Deletion behavior is controlled via `on_delete`:

- If an `Expense` is deleted, its child `ExpenseSplit` records cascade delete (`models.CASCADE`).
- If a `User` is deleted, expenses are protected (`models.PROTECT`) or reassigned to preserve historical financial ledger integrity, preventing ghost balance anomalies.

Category 5: Core Algorithms & Mathematical Logic

Q41: Explain the Greedy Debt Simplification Algorithm used in Pay-Together.

Answer: In a tour with N people, if every member pays debts to each individual creditor directly, there could be up to $N(N-1)/2 \approx O(N^2)$ separate transactions. The Greedy Debt Simplification algorithm minimizes this:

1. Calculate the **net balance** for every member:

$$\text{Net Balance}_i = \text{Total Paid by } i - \text{Total Consumed by } i$$

2. Members with $\text{Net} > 0$ are **Creditors**; members with $\text{Net} < 0$ are **Debtors**. Members with $\text{Net} = 0$ are settled.

3. Push creditors into a Max-Heap (or sorted list descending by balance), and debtors into a Min-Heap (sorted ascending by balance).

4. Greedily match the largest debtor with the largest creditor. Transfer amount:

$$\text{Amount} = \min(|\text{Debtor Balance}|, \text{Creditor Balance})$$

5. Update balances and remove fully settled members.

6. Repeat until all balances reach zero. This guarantees complete debt resolution in at most $N-1$ transactions.

Q42: What is the time complexity of the Greedy Debt Settlement algorithm?

Answer:

- Calculating net balances across M expense splits takes $O(M)$ time.
- Sorting or heap operations for N members take $O(N \log N)$ time.
- The matching loop executes at most $N-1$ times, with heap updates taking $O(\log N)$.

Overall time complexity is $O(M + N \log N)$, which runs in mere milliseconds for typical group travel sizes ($N \leq 100$).

Q43: Is the Greedy Debt Simplification algorithm mathematically guaranteed to find the absolute minimum number of transactions?

Answer: It guarantees resolution in at most $N-1$ transactions. While finding the *absolute* global minimum in all conceivable subset partitions is an NP-Hard problem (reducible to the Subset Sum Problem), the greedy heuristic achieves optimal or near-optimal solutions in polynomial time ($O(N \log N)$) without the exponential $O(2^N)$ computational cost.

Q44: What mathematical invariant must hold before running the settlement algorithm?

Answer: The zero-sum invariant:

$$\sum_{i=1}^N \text{Net Balance}_i = 0$$

Every rupee/dollar spent by one member is owed by others. If the sum does not equal zero (within a rounding tolerance of ± 0.01), the system flags a ledger corruption error before executing settlements.

Q45: How does the Equal Split algorithm handle odd amounts (e.g., \$100 divided equally among 3 people)?

Answer: Dividing 100 by 3 results in \$33.3333...\$. In Pay-Together:

- The base split is quantized to 2 decimal places: $\lfloor 100 / 3 \rfloor = 33.33$.
- Total accounted for: $33.33 \times 3 = 99.99$.
- The remainder of \$0.01 is allocated to the payer or the first participant. This ensures the sum of splits precisely equals 100.00 without loss or creation of pennies.

Q46: How does the Percentage Split mode work, and how is it validated?

Answer: Users assign a percentage to each participating member. The backend executes two validation rules:

1. Validation rule: $\sum (\text{percentages}) == 100.00\%$.
2. Share calculation: $\text{Share}_i = \text{quantize}(\text{Total}) \times \frac{\text{Percent}_i}{100}$.
3. Any minor rounding remainder ($< \$0.02$) is adjusted on the largest percentage share.

Q47: What is an "Edge-Triggered" Alert, and why is it used for spending limits?

Answer: An edge-triggered alert fires only when a variable crosses a defined threshold (a rising edge from below limit to above limit). If a user sets a limit of \$500 and their balance moves from \$480 to \$520, the alert triggers once. If subsequent transactions push the total to \$550 or \$600, duplicate alerts are suppressed, preventing notification spam.

Q48: What heuristic is implemented for smart expense categorization?

Answer: The system features a keyword-based heuristic classifier. As the user types the expense title (e.g., "Shell Fuel", "KFC Dinner", "Toll Booth", "Hotel Marriott"), regex keyword matchers automatically suggest the appropriate category (**Fuel** , **Food** , **Transport** , **Accommodation**) in real-time, speeding up entry.

Q49: What algorithm is used to detect runaway category spending?

Answer: A threshold monitoring routine runs after an expense is added:

$$\text{Ratio} = \frac{\sum \text{Expenses}_{\text{category}}}{\text{Tour Budget}}$$

If $\text{Ratio} > 0.30$ (category exceeds 30% of total tour budget), an analytical warning flag is generated on the tour summary page.

Q50: How does the system handle multi-payer expenses (e.g., two members splitting the initial bill payment)?

Answer: In multi-payer expenses, the total expense amount is composed of multiple credits (`ExpensePayer` mappings). In the net balance equation, both payers receive credit proportional to what they contributed upfront, while everyone's debt is computed based on their consumption share.

Category 6: Frontend Engineering, State & Offline Sync

Q51: What frontend libraries and styling technologies were used, and why?

Answer: The frontend is built using HTML5, modern vanilla JavaScript (ES6+), Tailwind CSS for styling, and Chart.js for data visualization. We chose modular Vanilla JavaScript over heavy frameworks like React/Angular to eliminate build-step complexity, ensure instant browser page loads, and integrate seamlessly with Django's server-rendered templates.

Q52: How does the Offline Queue mechanism work when a traveler loses internet connectivity?

Answer:

1. A JavaScript Service Worker or network event listener detects the `navigator.onLine === false` state.
2. When the user submits an expense, instead of failing, the payload is serialized into JSON and stored in browser `localStorage` / IndexedDB in an `offline_expense_queue`.
3. The UI displays an "Offline — Saved locally" badge.
4. When `window.addEventListener('online')` fires, the sync manager iterates through the queue and posts each item to the server sequentially.

Q53: How does the server prevent duplicate expenses if an offline sync request is transmitted multiple times?

Answer: Through **Idempotency Keys**. Each offline expense generated on the client receives a unique UUIDv4 token (`client_idempotency_key`). When the server receives an expense payload, it checks if a record with that key already exists. If it exists, the duplicate payload is safely acknowledged without creating a second database entry.

Q54: What visualizations are implemented with Chart.js?

Answer:

1. **Doughnut Chart:** Displays category-wise expense breakdown (Food, Fuel, Lodging, Activities).
2. **Horizontal Bar Chart:** Compares individual member spending vs. individual payments.
3. **Progress Bar / Gauge:** Shows current tour expenditure relative to total allocated budget.

Q55: How does the Digital Receipt Vault render images without slowing down page load times?

Answer: The application implements lazy loading (`loading="lazy"` attribute on `img` tags) and thumbnail caching. Clicking a thumbnail opens a responsive JavaScript modal lightbox showing the full-resolution receipt for inspection.

Q56: How is dynamic form behavior handled on the expense entry screen (e.g., toggling between Equal, Exact, and Percentage splits)?

Answer: A JavaScript module listens to the change event on the split type selector. It dynamically modifies the DOM:

- **Equal:** Disables input boxes, displaying auto-calculated shares.
- **Exact:** Displays currency input boxes beside each member with a live summation counter verifying $\sum \text{inputs} == \text{Total}$.
- **Percentage:** Displays percentage input boxes with a live counter verifying $\sum \% == 100\%$.

Q57: How is CSRF (Cross-Site Request Forgery) handled in AJAX/fetch calls in your JavaScript?

Answer: The JavaScript reads the `csrftoken` cookie value (or extracts it from `document.cookie`) and includes it in the HTTP request headers as `X-CSRFToken`:

```
````javascript
fetch('/api/expenses/', {
 method: 'POST',
 headers: {
 'Content-Type': 'application/json',
 'X-CSRFToken': getCookie('csrftoken')
 },
 body: JSON.stringify(payload)
});
````
```

Q58: What is Responsive Web Design, and how was it achieved in Pay-Together?

Answer: Responsive design ensures the web application functions flawlessly across diverse viewport dimensions (smartphones, tablets, laptops). Using Tailwind CSS utility classes (`sm:`, `md:`, `lg:`, `grid-cols-1`, `md:grid-cols-3`, `flex-wrap`), the layout reflows from multi-column desktop dashboards to single-column card stacks on mobile devices.

Q59: How does the system inform users of important events in real-time?

Answer: Through an asynchronous notification drawer and toast popups. When a user logs in or performs an action, an unread notification counter badge is fetched via an API call and displayed in the navigation bar.

Q60: How does Pay-Together ensure accessibility (a11y) standards?

Answer: All interactive buttons and inputs have semantic labels (`aria-label`), clear focus indicators, high color contrast ratios conforming to WCAG 2.1 AA standards, and keyboard navigation support.

Category 7: Security, Authentication & Access Control

Q61: What password hashing algorithm does Pay-Together use, and why is plain text storage unacceptable?

Answer: Pay-Together uses Django's default PBKDF2 algorithm with a SHA-256 hash and 600,000+ iterations (or Argon2/bcrypt if configured). Plain text storage violates fundamental security principles; if the database is leaked, plain text passwords expose user credentials. Salted and iterated password hashes make rainbow-table and brute-force attacks computationally infeasible.

Q62: How does Pay-Together enforce Role-Based Access Control (RBAC)?

Answer: Permissions are enforced at both the View and Object level:

- **View Level:** `LoginRequiredMixin` ensures unauthenticated users cannot access internal tour paths.
- **Object Level:** Custom permission decorators verify that the `request.user` is a registered member of the specific `Tour` referenced in the URL parameter. Non-members receive an HTTP 403 Forbidden response.

Q63: What is Cross-Site Scripting (XSS), and how does the application defend against it?

Answer: XSS occurs when malicious JavaScript code is injected into input fields and executed in other users' browsers. Django templates automatically HTML-escape all variables by default (converting